DevOps 03, Opgaver

Opg. 01, Build + Run (fra Dockerfile til container)

Formål

At træne hele kæden: `Dockerfile -> build -> image -> run -> container`

Opgave

Lav en Dockerfile til en simpel app (du bestemmer udviklingssproget) der kan starte og svare på HTTP requests.

- Brug `WORKDIR`
- Brug `COPY` og `RUN` til dependencies og opdateringer
- Brug `ENTRYPOINT`
- Kør containeren med nødvendig exposure

Opgaven er løst når

- `docker build -t <name>:<tag> .` bygger et image uden fejl
- `docker run -p ...` starter containeren
- Du kan tilgå appen i browseren på localhost
- `docker ps` viser containeren

Opg. 02, Cache-first struktur (målbar effekt)

Formål

At forstå ud udnytte cache ved korrekt Dockerfile struktur

Opgave

Tag din Dockerfile fra opg. 01 og lav to versioner:

1. "dårlig cache" (monitisk copy) (cache efter kode copy)
2. "cache first" (copy manifest først)

Krav:

- Byg begge images mindst 2 gange hver
- Foretag en ændring i i applikationskoden (ikke dependencies) og byg igen
- Notér forskellen i build output (hvilke steps kører igen, og hvor lang tid tager det)

Opgaven er løst når

- Du har to Dockerfiles (fx monolit.Dockerfile og cachefirst.Dockerfile)
- Du kan forklare hvorfor den ene performer bedre end den anden
- Du kan vise i build output at nogle steps er cached

Opg. 03, Non-root user (least privilege)

Formål

At køre appliaktionen uden root i containeren.

Opgave

Tilføj non-root user i Dockerfile og få appen til at køre som denne user.

Krav:

- Opret user/group og brug `USER` instruksen i Dockerfile
- Hvis appen skal skrive filer (logs/uploads), så håndter nødvendige permissions (Linux)

Opgaven er løst når

- Contianeren kører din app som non-root
- `id` kommandoen i containeren viser den non-root user
- Appen stadig virker

Opg. 04, Multi-stage build

Formål

At adskille build-stage og runtime-stage og reducere image size

Opgave

Lav en multi-stage Dockerfile

Med udgangspunkt i din eksisterende Dockerfile, lav en multi-stage version hvor du har:

- En "build" stage der bruger et SDK image til at bygge appen
- En "runtime" stage der bruger et mindre image og kun kopierer det nødvendige fra build stage
- Byb og sammenlign image size (før/efter)
- Kør containeren og verificér at den virker

Opgaven er løst når

- Du har en multi-stage Dockerfile
- Du kan vise docker images size før og efter
- Runtime containeren fungerer

Opg. 05, Images som artifacts

Formål

At arbejde med image-tags som versionssyring af artifacts

Opgave

Byg samme app i mindst 3 versioner:

- Lav en ændring i koden for hver version (fx en tekstændring i UI)
- Byg et image for hver version med semver tags (fx `:1.0.0`, `:1.1.0`, `:2.0.0`)
- Kør containeren for hver version og verificér ændringen i UI

Opgaven er løst når

- Du har mindst 3 images med forskellige tags
- Du kan forklare semver versioning
- Du kan starte hver version og se forskellen i output
- Du kan forklare hvorfor "rollback" er nemt med tags

Opg.06, Docker Registry

Sammenfatningsopgave

Formål

At forstå brugen af Docker Registry til at hoste og versionere images

Opgave

- Kør en lokal Docker Registry
- Tag dit image således at det kan pushes til dit registry
- Push image til registry
- Pull image fra registry og kør containeren
- Gør ovenstående med forskellige versioner
- Valider at versioner af images ligger i registry med brug af registry API

[Optional]

- Sæt auth op på registry og test push/pull med auth
- Login til registry
- Push/pull med auth
- Opret faktisk server med registry og test fra en anden maskine (fx cloud VM) (denne koster penge og vil være selvbetalt hvis du ønsker at gennemføre opgaven)

Opgaven er løst når

- Du har en lokal registry kørende
- Du kan push/pull images til/fra registry
- Du kan starte container fra pulled image
- Du kan se versioner i registry
- [Optional] Du har auth på registry og kan push/pull med auth